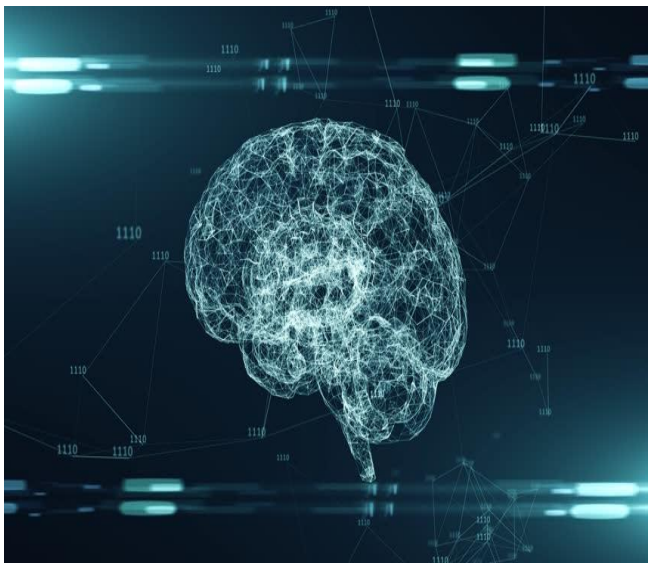

AIML VITYARTHI PROJECT

NAME- GORANSHI AGARWAL

REG NO.- 25BAI11505

1. INTRODUCTION TO THE PROBLEM

This project implements a solution to the 8-puzzle problem using the **A* search algorithm**, a fundamental concept in Artificial Intelligence.

The 8-puzzle consists of a 3×3 grid with tiles from 1 to 8 and one blank space (represented by 0). The objective is to rearrange the tiles from an initial state to a goal state using the minimum number of moves (optimal solution).

The program takes input for the initial state, checks whether the puzzle is solvable, and then applies the A* algorithm with the Manhattan heuristic to find the optimal solution. It outputs the number of steps required along with the step-by-step transformation from the initial state to the goal state.

2. PROBLEM STATEMENT

The 8-puzzle consists of: A **3×3 grid**

- 8 numbered tiles (1 to 8)
- 1 blank space (represented by 0)

Objective:

To rearrange the tiles from an initial state to a goal state in the minimum number of moves

Rules:

- Only the 0 (blank space) can be moved
- Moves allowed: Up, Down, Left, Right
- Each move has equal cost
- Not all input puzzles are solvable

3. SIGNIFICANCE

This problem helps us to gain knowledge about various AI algorithms such as:

- Introduce to state space search techniques
- understanding heuristic-based algorithms
- Demonstrates optimal pathfinding
- Forms a base for real-world applications such as:

    a. Navigation systems
    b. Game AI
    c. Robotics

4. CONCEPTS USED:

a. A* search Algorithm

The A* algorithm is an informed search algorithm used in Artificial Intelligence to find the shortest and most optimal path from a starting point to a goal.

A* works by choosing the best node (state) based on a cost function:

F(n) = g(n) + h(n)

Where:

- g(n) = Actual cost from start to current node
- h(n) = Estimated cost from current node to goal (heuristic)
- f(n) = Total estimated cost

Advantages:

- Always finds the shortest path
- Gives best possible result if heuristic is correct
- Guarantees a solution if it exists
- Uses heuristic and avoids unnecessary paths
- Uses Intelligent Decision Making

b. Heuristic Function

The Manhattan Distance heuristic is used:

- It calculates the distance of each tile from its correct position
- Total distance = sum of all tile distances
- This heuristic improves efficiency and guarantees optimal results.

5. PYTHON CODE

```
from queue import PriorityQueue

goalstate = [[1,2,3],
             [4,5,6],
             [7,8,0]]

moves = [(-1,0),(1,0),(0,-1),(0,1)]

def findblank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def heuristic(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                x = (value - 1) // 3
                y = (value - 1) % 3
                distance += abs(i - x) + abs(j - y)
    return distance

# Convert to tuple
def totuple(state):
    return tuple(tuple(row) for row in state)

def print_stack(state):
    for row in state:
        print(row)
    print()

def solvable(state):
    flat = [num for row in state for num in row if num != 0]
    inversions = 0
    for i in range(len(flat)):
        for j in range(i+1, len(flat)):
            if flat[i] > flat[j]:
```

```python
                inversions += 1
    return inversions % 2 == 0

# A* Solver
def solvepuzzle(start):
    pq = PriorityQueue()
    pq.put((0, start, 0, []))  # (f, state, g, path)

    visited = set()

    while not pq.empty():
        f, state, g, path = pq.get()

        if state == goalstate:
            print("Solution Found!")
            print("Steps:", g)
            print("\nPath:\n")
            for step in path + [state]:
                print_stack(step)
            return

        visited.add(totuple(state))

        x, y = findblank(state)

        for dx, dy in moves:
            nx, ny = x + dx, y + dy

            if 0 <= nx < 3 and 0 <= ny < 3:
                new_state = [row[:] for row in state]
                new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

                if totuple(new_state) not in visited:
                    h = heuristic(new_state)
                    pq.put((g + 1 + h, new_state, g + 1, path + [state]))

    print("No solution found")

# 🔷 TAKE INPUT FROM USER
print("Enter the 8-puzzle (use 0 for blank):")

start_state = []
for i in range(3):
    row = list(map(int, input(f"Enter row {i+1} (space separated): ").split()))
    start_state.append(row)
```

```python

print("\nInitial State:")
print_stack(start_state)

# Check solvability
if solvable(start_state):
    solvepuzzle(start_state)
else:
    print("This puzzle is NOT SOLVABLE")
```

6. WORKING

A. Take Initial Input from User

- The program starts by taking a 3×3 matrix input
- User enters numbers from 0 to 8
- Where 0 represents the blank space
- 0 can only be moved

B. Check Solvability using Inversion Count

- Before solving, the program checks if the puzzle **can be solved**
- It counts **inversions** (wrong order pairs)

Rules

Even inversions are **Solvable**

Odd inversions are **Not solvable**

C. Initialize Priority Queue

- A priority queue is created to store states
- States are arranged from lowest to highest cost
- This ensures the algorithm always picks the best state also called optimal solution

D. Generate Possible Moves

- From the current state, all valid moves are generated:

  - Up
  - Down
  - Left
  - Right

E. Calculate Cost Function for Each State

The cost is calculated using:

$$f(n) = g(n) + h(n)$$

- g(n) = number of moves so far
- h(n) = heuristic cost

F. Select State with Lowest Cost

- From the priority queue, the state with lowest f(n) is selected
- This is the most suitable state

G. . Repeat Until Goal State is Reached

- Steps 4–6 are repeated:

  - Generate new states
  - Calculate cost
  - Select the best state out of them

- Loop continues until:

  - Goal state is found
    OR
  - No more states available

H. Display Solution Path

- Once the goal is reached:
    - Print the number of steps
    - Show step-by-step states as solution

7. SAMPLE INPUT

```
PS C:\Users\pallavi\AppData\Local\Programs\Microsoft VS Code> & C:\Us
top/Python/from queue import PriorityQueue.py"
Enter the 8-puzzle (use 0 for blank):
Enter row 1 (space separated): 1 2 3
Enter row 2 (space separated): 4 5 6
Enter row 3 (space separated): 0 7 8
```

The program requires the user to provide the initial configuration of the 8-puzzle.

Input should be in format of

- A 3×3 matrix
- Values range from 0 to 8
- Each number appears exactly once
- 0 represents the blank space

Rules

- User enters values row by row
- Spaces separate each numbers

8. SAMPLE OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE

Initial State:
[1, 2, 3]
[4, 5, 6]
[0, 7, 8]

Solution Found!
Steps: 2

Path:
```

```
PROBLEMS    OUTPUT    DEBUG

[1, 2, 3]
[4, 5, 6]
[0, 7, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TER

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

.

For output

The first code will check whether the problem is solvable or not via the solvability check.

- If not, the result will be displayed as " This problem is NOT SOLVABLE"
- If yes, then it will solve the problem step-by-step with all possible methods and find the optimal solution and path cost

9. FUTURE ENHANCEMENT:

1. Graphical User Interface (GUI)

A. Convert CLI program into a visual interactive interface

B. Use tools like:

   a. Tkinter (Python)

C. Show puzzle as clickable tiles

D. Makes the project more user-friendly and attractive

2. Animation of Solution Steps

   A. Display tile movements step-by-step visually

   B. Animate transitions between states

   C. It Helps in better understanding and presentation

3. Extend to Larger Problems

   A. Expand project to: 15-puzzle (4×4 grid)

10. REFERENCES:

1. GeeksforGeeks - A* Search Algorithm
2. Wikipedia - A* Search Algorithm
3. Khan Academy (Youtube)